



มหาวิทยาลัยขอนแก่น

วิทยา จวิทยา ปัญญา

KHON KAEN UNIVERSITY



Normalization (Part 2)

Assoc. Prof. Dr. Kanda Runapongsa Saikaew
Faculty of Engineering
Khon Kaen University



คณะวิศวกรรมศาสตร์ มหาวิทยาลัยขอนแก่น
FACULTY OF ENGINEERING KHON KAEN UNIVERSITY

Agenda

- Normal Forms
- Functional Dependency Theory
- Algorithms for Decomposition using Functional Dependencies
- Atomic Domains and First Normal Form
- Database-Design Process



Boyce-Codd Normal Form

- A relation R is in Boyce-Codd Normal Form (BCNF) if and only if for every non-trivial functional dependency $X \rightarrow Y$ in R , X is a superkey of R .
- In other words:
 - For all functional dependencies in R of the form $X \rightarrow Y$, where X and Y are sets of attributes, X must be a superkey.
 - This means that X should be able to uniquely determine all other attributes in the relation.
- Key points:
 - It eliminates all anomalies resulting from functional dependencies



BCNF Relation

- Consider a relation: Student(StudentID, Name, Major)
Functional Dependencies:
 - StudentID $\rightarrow$ Name, Major
 - Name $\rightarrow$ StudentID, Major
- This relation is in BCNF because:
 - StudentID is a candidate key (it determines all other attributes)
 - Name is also a candidate key (it determines all other attributes)
 - There are no non-trivial functional dependencies where the left side is not a superkey.



Relation Not in BCNF

- Consider a relation: Course(CourseID, Professor, TextBook) Functional Dependencies:
 - CourseID, Professor $\rightarrow$ TextBook
 - Professor $\rightarrow$ TextBook
- This relation is not in BCNF because:
 - The functional dependency Professor $\rightarrow$ TextBook violates BCNF
 - Professor is not a superkey as it doesn't uniquely determine CourseID
- To achieve BCNF, we would decompose this relation into:
 - R1(Professor, TextBook)
 - R2(CourseID, Professor)



BCNF Decomposition

- Original Relation: Enrollment(StudentID, CourseID, Professor, Grade)
Functional Dependencies:
 - StudentID, CourseID $\rightarrow$ Grade
 - CourseID $\rightarrow$ Professor
- This relation is not in BCNF because CourseID $\rightarrow$ Professor, and CourseID is not a superkey.
- BCNF Decomposition:
 - R1(StudentID, CourseID, Grade)
 - R2(CourseID, Professor) $\bowtie$
- Now both relations are in BCNF:
 - In R1, (StudentID, CourseID) is the only non-trivial functional dependency, and it's a superkey.
 - In R2, CourseID $\rightarrow$ Professor, and CourseID is a superkey for R2.



BCNF and Dependency Preservation

- Consider a relation R with the following attributes and functional dependencies:
 - R(StudentID, CourseID, Professor)
 - FDs:
 - StudentID, CourseID $\rightarrow$ Professor
 - Professor $\rightarrow$ CourseID
 - In this scenario:
 - A student can only take a course with one professor
 - A professor teaches only one course
1. Current schema violates BCNF: The FD Professor $\rightarrow$ CourseID violates BCNF because Professor is not a superkey.
 2. BCNF Decomposition: R1(StudentID, Professor) R2(Professor, CourseID)
 3. This decomposition achieves BCNF but loses the dependency StudentID, CourseID $\rightarrow$ Professor.
 - To check this dependency, we now need to join R1 and R2, which is computationally expensive.



Third Normal Form

- A relation R is in Third Normal Form (3NF) if for every non-trivial functional dependency $X \rightarrow A$ in R, at least one of the following conditions holds:
 1. X is a superkey of R, or
 2. A is a prime attribute (i.e., A is part of some candidate key of R)
- In other words, 3NF requires that every non-prime attribute of R is:
 - Fully functionally dependent on every key of R, and
 - Not transitively dependent on any key of R.



Difference between 3NF and BCNF (1/2)

- Definition:
 - 3NF allows non-prime attributes to be functionally dependent on prime attributes.
 - BCNF is stricter and requires that for every non-trivial functional dependency $X \rightarrow A$, X must be a superkey.
- Dependency on prime attributes:
 - 3NF allows dependencies where a non-prime attribute depends on part of a composite key (prime attribute).
 - BCNF does not allow any attribute to be dependent on anything less than a full candidate key.



Difference between 3NF and BCNF (2/2)

- Transitive dependencies:
 - 3NF eliminates most, but not all, transitive dependencies.
 - BCNF eliminates all transitive dependencies.
- Anomalies:
 - 3NF may still have some update anomalies in certain cases.
 - BCNF eliminates all modification anomalies.
- Dependency preservation:
 - 3NF always guarantees a dependency-preserving decomposition.
 - BCNF does not always guarantee a dependency-preserving decomposition.



3NF vs BCNF Example

- Consider a relation $R(A, B, C)$ with functional dependencies:
 - $AB \rightarrow C$
 - $C \rightarrow B$
- This relation is in 3NF because:
 - AB is a candidate key
 - $C \rightarrow B$ doesn't violate 3NF as B is a prime attribute (part of the candidate key AB)
- However, this relation is not in BCNF because of the functional dependency $C \rightarrow B$, where C is not a superkey.
- To achieve BCNF, we would decompose R into: $R_1(A, C)$ and $R_2(C, B)$
- This decomposition achieves BCNF but doesn't preserve the dependency $AB \rightarrow C$.



Redundancy in 3NF

- Consider the schema R below, which is in 3NF

- $R = (J, K, L)$
- $F = \{JK \rightarrow L, L \rightarrow K\}$
- And an instance table:

J	L	K
j_1	l_1	k_1
j_2	l_1	k_1
j_3	l_1	k_1
null	l_2	k_2

- What is wrong with the table?
 - Repetition of information
 - Need to use null values (e.g., to represent the relationship l_2, k_2 where there is no corresponding value for J)



Comparison of BCNF and 3NF

- Advantages to 3NF over BCNF.
 - It is always possible to obtain a 3NF design without sacrificing losslessness or dependency preservation.
 - BCNF, while eliminating more anomalies than 3NF, sometimes requires sacrificing dependency preservation to achieve its stricter form of normalization.
- Disadvantages to 3NF.
 - 3NF may still allow some update anomalies that BCNF eliminates.
 - Specifically, 3NF can have modification anomalies related to non-prime attributes dependent on part of a composite key.



Comparison of BCNF and 3NF Example (1/3)

- Consider a relation R(Student_ID, Course_ID, Professor_ID, Professor_Name, Department) With functional dependencies:
 - Student_ID, Course_ID $\rightarrow$ Professor_ID
 - Professor_ID $\rightarrow$ Professor_Name, Department
- This relation is in 3NF because:
 - (Student_ID, Course_ID) is a candidate key
 - Professor_ID $\rightarrow$ Professor_Name, Department doesn't violate 3NF as Professor_ID is a prime attribute
- However, it's not in BCNF because Professor_ID determines non-prime attributes but is not a superkey.



Comparison of BCNF and 3NF Example (2/3)

- Disadvantages of this 3NF design:
 1. Update Anomaly: Changing a professor's name or department requires updating multiple tuples
 2. Insertion Anomaly: Can't add a new professor without an associated course
 3. Deletion Anomaly: Deleting all courses for a professor loses the professor's information.
 4. Data Redundancy: Professor's name and department are repeated for each course they teach.



Comparison of BCNF and 3NF Example (3/3)

- BCNF Decomposition:
 - R1(Student_ID, Course_ID, Professor_ID)
 - R2(Professor_ID, Professor_Name, Department)
- This BCNF design eliminates these issues, providing a cleaner and more efficient structure.
- In conclusion
 - While 3NF is often sufficient and guarantees dependency preservation, BCNF offers a more refined and anomaly-free design.
 - The choice between 3NF and BCNF often depends on specific requirements of the database application, balancing the need for dependency preservation with the desire for minimal redundancy and anomaly elimination.



Goals of Normalization

- Let R be a relation scheme with a set F of functional dependencies.
- Decide whether a relation scheme R is in “good” form.
- In the case that a relation scheme R is not in “good” form, need to decompose it into a set of relation scheme $\{R_1, R_2, \dots, R_n\}$ such that:
 - Each relation scheme is in good form
 - The decomposition is a lossless decomposition
 - Preferably, the decomposition should be dependency preserving.



How good is BCNF?

- There are database schemas in BCNF that do not seem to be sufficiently normalized
- Consider a relation
inst_info (*ID*, *child_name*, *phone*)
 - where an instructor may have more than one phone and can have multiple children
 - Instance of *inst_info*

<i>ID</i>	<i>child_name</i>	<i>phone</i>
99999	David	512-555-1234
99999	David	512-555-4321
99999	William	512-555-1234
99999	William	512-555-4321

- There are no non-trivial functional dependencies and therefore the relation is in BCNF
- Insertion anomalies – i.e., if we add a phone 981-992-3443 to 99999, we need to add two tuples
(99999, David, 981-992-3443)
(99999, William, 981-992-3443)



Higher Normal Forms

- It is better to decompose *inst_info* into:
 - *inst_child*:

<i>ID</i>	<i>child_name</i>
99999	David
99999	William

- *inst_phone*:

<i>ID</i>	<i>phone</i>
99999	512-555-1234
99999	512-555-4321

- This suggests the need for higher normal forms, such as Fourth Normal Form (4NF), which we shall see later



Functional-Dependency Theory Roadmap

- We now consider the formal theory that tells us which functional dependencies are implied logically by a given set of functional dependencies.
- We then develop algorithms to generate lossless decompositions into BCNF and 3NF
- We then develop algorithms to test if a decomposition is dependency-preserving



Closure of a Set of Functional Dependencies

- Given a set F set of functional dependencies, there are certain other functional dependencies that are logically implied by F .
 - If $A \rightarrow B$ and $B \rightarrow C$, then we can infer that $A \rightarrow C$
- The set of **all** functional dependencies logically implied by F is the **closure** of F .
- We denote the *closure* of F by F^+ .



Closure of a Set of Functional Dependencies

- We can compute F^+ , the closure of F , by repeatedly applying **Armstrong's Axioms**:
 - **Reflexive rule**: if $\beta \subseteq \alpha$, then $\alpha \rightarrow \beta$
 - **Augmentation rule**: if $\alpha \rightarrow \beta$, then $\gamma \alpha \rightarrow \gamma \beta$
 - **Transitivity rule**: if $\alpha \rightarrow \beta$, and $\beta \rightarrow \gamma$, then $\alpha \rightarrow \gamma$
- These rules are
 - **Sound** -- generate only functional dependencies that actually hold, and
 - **Complete** -- generate all functional dependencies that hold.



Example of F^+

- $R = (A, B, C, G, H, I)$
 $F = \{ A \rightarrow B$
 $A \rightarrow C$
 $CG \rightarrow H$
 $CG \rightarrow I$
 $B \rightarrow H \}$
- Some members of F^+
 - $A \rightarrow H$
 - by transitivity from $A \rightarrow B$ and $B \rightarrow H$
 - $AG \rightarrow I$
 - by augmenting $A \rightarrow C$ with G , to get $AG \rightarrow CG$
 and then transitivity with $CG \rightarrow I$
 - $CG \rightarrow HI$
 - by augmenting $CG \rightarrow I$ to infer $CG \rightarrow CGI$,
 and augmenting of $CG \rightarrow H$ to infer $CGI \rightarrow HI$, and then transitivity



Closure of Functional Dependencies (Cont.)

- Additional rules:
 - **Union rule:** If $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds, then $\alpha \rightarrow \beta \gamma$ holds.
 - **Decomposition rule:** If $\alpha \rightarrow \beta \gamma$ holds, then $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds.
 - **Pseudo transitivity rule:** If $\alpha \rightarrow \beta$ holds and $\gamma \beta \rightarrow \delta$ holds, then $\alpha \gamma \rightarrow \delta$ holds.
- The above rules can be inferred from Armstrong's axioms.



Procedure for Computing F^+

- To compute the closure of a set of functional dependencies F :

$$F^+ = F$$

repeat

for each functional dependency f in F^+

 apply reflexivity and augmentation rules on f

 add the resulting functional dependencies to F^+

for each pair of functional dependencies f_1 and f_2 in F^+

if f_1 and f_2 can be combined using transitivity

then add the resulting functional dependency to F^+

until F^+ does not change any further

- **NOTE:** We shall see an alternative procedure for this task later



Closure of Attribute Sets

- Given a set of attributes α , define the **closure** of α **under** F (denoted by α^+) as the set of attributes that are functionally determined by α under F
- Algorithm to compute α^+ , the closure of α under F
result := α ;
while (changes to *result*) **do**
 for each $\beta \rightarrow \gamma$ **in** F **do**
 begin
 if $\beta \subseteq \text{result}$ **then** *result* := *result* $\cup \gamma$
 end



Example of Attribute Set Closure

- $R = (A, B, C, G, H, I)$
- $F = \{A \rightarrow B$
 $A \rightarrow C$
 $CG \rightarrow H$
 $CG \rightarrow I$
 $B \rightarrow H\}$
- $(AG)^+$
 1. $result = AG$
 2. $result = ABCG$ ($A \rightarrow C$ and $A \rightarrow B$)
 3. $result = ABCGH$ ($CG \rightarrow H$ and $CG \subseteq AGBC$)
 4. $result = ABCGHI$ ($CG \rightarrow I$ and $CG \subseteq AGBCH$)
- Is AG a candidate key?
 1. Is AG a super key?
 1. Does $AG \rightarrow R$? == Is $R \supseteq (AG)^+$
 2. Is any subset of AG a superkey?
 1. Does $A \rightarrow R$? == Is $R \supseteq (A)^+$
 2. Does $G \rightarrow R$? == Is $R \supseteq (G)^+$
 3. In general: check for each subset of size $n-1$



Uses of Attribute Closure

There are several uses of the attribute closure algorithm:

- Testing for superkey:
 - To test if α is a superkey, we compute α^+ and check if α^+ contains all attributes of R .
- Testing functional dependencies
 - To check if a functional dependency $\alpha \rightarrow \beta$ holds (or, in other words, is in F^+), just check if $\beta \subseteq \alpha^+$.
 - That is, we compute α^+ by using attribute closure, and then check if it contains β .
 - Is a simple and cheap test, and very useful
- Computing closure of F
 - For each $\gamma \subseteq R$, we find the closure γ^+ , and for each $S \subseteq \gamma^+$, we output a functional dependency $\gamma \rightarrow S$.



Denormalization for Performance

- May want to use non-normalized schema for performance
- For example, displaying *prereqs* along with *course_id*, and *title* requires join of *course* with *prereq*
- Alternative 1: Use denormalized relation containing attributes of *course* as well as *prereq* with all above attributes
 - faster lookup
 - extra space and extra execution time for updates
 - extra coding work for programmer and possibility of error in extra code
- Alternative 2: use a materialized view defined a *course prereq*
 - Benefits and drawbacks same as above, except no extra coding work for programmer and avoids possible errors



First Normal Form (1NF)

- A relation is in First Normal Form (1NF) if and only if:
 1. It contains only atomic (indivisible) values
 2. There are no repeating groups or arrays
- Key Concepts:
 1. Atomic Values: Each column should contain a single value, not a set of values or a nested record
 2. No Repeating Groups: There should be no columns that contain similar types of information for a single record.



Example1: Student Information (1/2)

- Not in 1NF:

StudentInfo(StudentID, Name, Courses)

StudentID	Name	Courses
1	Alice	Math, Physics, Chemistry
2	Bob	Biology, Chemistry
3	Charlie	Physics, Computer Science

This violates 1NF because the Courses column contains multiple values.



Example1: Student Information (2/2)

- Converted to 1NF: StudentCourses(StudentID, Name, Course)

StudentID	Name	Course
1	Alice	Math
1	Alice	Physics
1	Alice	Chemistry
2	Bob	Biology
2	Bob	Chemistry
3	Charlie	Physics
3	Charlie	Computer Science



Second Normal Form (2NF)

- A relation is in Second Normal Form (2NF) if and only if:
 1. It is in First Normal Form (1NF)
 2. Every non-prime attribute is fully functionally dependent on the entire primary key
- Key Concepts:
 1. Prime Attribute: An attribute that is part of any candidate key.
 2. Non-prime Attribute: An attribute that is not part of any candidate key.
 3. Full Functional Dependency: An attribute is fully functionally dependent on a set of attributes if it depends on the entire set and not on any subset of it.
 4. Partial Dependency: A functional dependency where an attribute depends on only part of a composite key.



Not in 2NF

- Not in 2NF: Enrollment(StudentID, CourseID, CourseName, StudentName, Grade) Primary Key: (StudentID, CourseID)
- Functional Dependencies:
 - StudentID, CourseID $\rightarrow$ Grade, StudentID $\rightarrow$ StudentName, CourseID $\rightarrow$ CourseName
- This relation violates 2NF because:
 - StudentName depends only on StudentID (part of the primary key)
 - CourseName depends only on CourseID (part of the primary key)



Converted to 2NF

R1. StudentCourse(StudentID, CourseID, Grade)

R2. Student(StudentID, StudentName)

R3. Course(CourseID, CourseName)

R1. StudentID, CourseID $\Rightarrow$ Grade

R2. StudentID $\Rightarrow$ StudentName

R3. CourseID $\Rightarrow$ CourseName



Fourth Normal Form (4NF)

- A relation R is in 4NF if, for every non-trivial multivalued dependency $X \twoheadrightarrow Y$ in R, X is a superkey of R
- Key Concepts:
 - Multivalued Dependency (MVD)
 - Denoted as $X \twoheadrightarrow Y$
 - It means for each X value, there is a set of Y values, and this set is independent of the values of other attributes
 - Non-trivial MVD
 - An MVD $X \twoheadrightarrow Y$ is non-trivial if Y is neither empty nor a subset of X, and $X \cup Y$ is not the entire set of attributes



Why 4NF is Necessary

- Differences between BCNF and 4NF
 - BCNF deals with functional dependencies.
 - 4NF deals with multivalued dependencies, which are a more general form of dependency.
- Why 4NF is necessary
 - Every relations in BCNF can suffer from redundancy when there are multivalued dependencies
 - 4NF eliminates these redundancies



Not in 4NF

Consider a relation: Student_Clubs_Skills(StudentID, Club, Skill)

Assume:

- A student can be in multiple clubs
- A student can have multiple skills
- The clubs a student joins are independent of their skills

This relation satisfies BCNF (assuming StudentID is the key),

but it's not in 4NF because of these multivalued dependencies:

StudentID $\twoheadrightarrow$ Club

StudentID $\twoheadrightarrow$ Skill

StudentID	Club	Skill
1	Chess	Java
1	Chess	Python
1	Debate	Java
1	Debate	Python



Converted to 4NF

- Student_Clubs(StudentID, Club)
- Student_Skills(StudentID, Skill)
- This decomposition eliminates the redundancy and achieves 4NF.
- 4NF Decomposition Algorithm:
 1. Start with a relation R.
 2. For each non-trivial MVD $X \twoheadrightarrow Y$ that violates 4NF:
 - Create two relations: R1(X, Y) and R2(X, Z), where Z is all attributes not in X or Y.
 - Repeat until all relations are in 4NF



Conclusions (1/2)

- Normalization is a critical process in database design for reducing redundancy and improving data integrity
- Different normal forms (1NF, 2NF, 3NF, BCNF, 4NF) address various aspects of data organization and dependencies
- BCNF provides the strictest form of normalization for functional dependencies but may not always preserve all dependencies
- 3NF offers a balance between normalization and dependency preservation



Conclusions (2/2)

- 4NF addresses multi-valued dependencies not handled by lower normal forms
- The choice of normal form depends on specific application requirements and trade-offs
- In some cases, denormalization may be considered for performance optimization, but it comes with trade-offs
- The database design process should aim for the highest practical normal form while considering performance, usability, and data integrity



References

- Avi Silberschatz, Henry F. Korth, S. Sudarshan. Database System Concepts [Internet]. New Haven: Yale University; [cited 2024 Sep 10]. Available from: <https://codex.cs.yale.edu/avi/db-book/slides-dir/index.html>
- Anthropic. Claude AI [Internet]. San Francisco: Anthropic; 2024 [cited 2024 Sep 22]. Available from: <https://www.anthropic.com>

